PROGRAMMING & CS FUNDAMENTALS

Type Systems Visual Guide

Static vs dynamic, inference, generics — what types prove before your code runs.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Static vs Dynamic Typing

WHEN are type errors caught: compile time or run time?

Static
checked at■compile time

Dynamic
checked at■run time

```
# static (Rust):   let x: i32 = 'hi';   // COMPILE error, ships never
# dynamic (Python): x = 'hi'; x + 1     // RUNTIME TypeError, ships maybe
```

- Static: errors caught early, IDE autocompletion, zero-cost abstractions — at the price of ceremony.
- Dynamic: fast prototyping, metaprogramming — at the price of runtime surprises and tests-as-types.
- Strong vs weak is SEPARATE: strong = no silent coercion; weak = '5' + 5 might 'work'.

STEP 2/6

Type Inference: Best of Both

Static checking WITHOUT the annotations — the compiler fills them in.

```
let x = 42;            // Rust: inferred i32, still static!
auto y = compute();    // C++: deduced
val z = List(1,2,3)    // Scala/Kotlin: full-program inference

# Hindley-Milner: infers the MOST GENERAL type automatically
```

- Inference != dynamic: types are still fixed at compile time, just unwritten.
- Local inference (var/auto) is safe; global inference can produce cryptic errors.
- Annotate PUBLIC APIs anyway: inference is for locals, documentation is for humans.
- TypeScript: gradual inference over JavaScript — types appear where they pay off.

STEP 3/6

Generics: Types with Parameters

Write once, type-check for every contained type.

```
# List of T: one implementation, checked per instantiation
def first(items: list[T]) -> T: return items[0]
first([1, 2])      # T = int, returns int
first(['a'])        # T = str, returns str
# vs object/dynamic: no casts, no runtime surprises
```

- Parametric polymorphism: the algorithm doesn't care WHAT T is — only that it's consistent.
- Type erasure (Java): generics vanish at runtime — no List-of-int at the bytecode level.
- Reified (C#, Rust): types survive to runtime — reflection and specialization work.
- Bounds: 'T must be Comparable' — constrain the parameter where the algorithm needs it.

STEP 4/6

Variance: Subtyping Through Generics

If Cat is an Animal, is a List of Cat a List of Animal?

Covariant

List of Cat IS a List of Animal — safe for READS (producers)

Contravariant

reversed: sinks/consumers — safe for WRITES (comparators)

Invariant

List of Cat is NOT List of Animal — mutable collections: both read+write

```
# why mutable lists are invariant:
# if List[Cat] were a List[Animal]: append(Dog()) compiles... boom.
```

Mnemonic: producers are covariant (out), consumers contravariant (in) — 'PECS: producer extends, consumer super'.

STEP 5/6

Gradual Typing Pitfalls

Half-typed codebases have half the guarantees — mind the gaps.

- Any/Dynamic is contagious: one untyped boundary poisons inference downstream.
- Type assertions (casts) are YOU overriding the checker — wrong casts are runtime lies.
- None/null: the billion-dollar mistake — Optional types make absence EXPLICIT.
- Structural (duck) vs nominal (named) typing: 'has these methods' vs 'IS this class'.
- Soundness vs completeness: some checkers reject valid programs to stay sound — learn the trade.
- Migrate strategically: type the boundaries and hot paths first, not the tests.

STEP 6/6

Types as Design Tools

Let the checker carry what tests can't.

- Make illegal states unrepresentable: enums over booleans, newtypes over raw ints.
- Parse, don't validate: convert untrusted input ONCE into strong types at the boundary.
- Exhaustiveness: match over enums warns on missing cases — new variant? compiler finds every site.
- Phantom types: encode permissions/units in the type, zero runtime cost.
- Property tests + types: types prove the shape, tests probe the behavior — complementary.
- The type signature is documentation that never rots — read it before the body.

STEP ✓I✓

Key Takeaways

Type Systems — Remember

- Static catches errors at compile time; dynamic at run time — strong/weak is a separate axis.
- Inference gives static safety without annotation noise — but annotate public APIs.
- Generics (List of T): one implementation, checked per type; erasure vs reification matters.
- Variance: producers covariant, consumers contravariant, mutable collections invariant.
- Make illegal states unrepresentable — types are executable documentation.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev